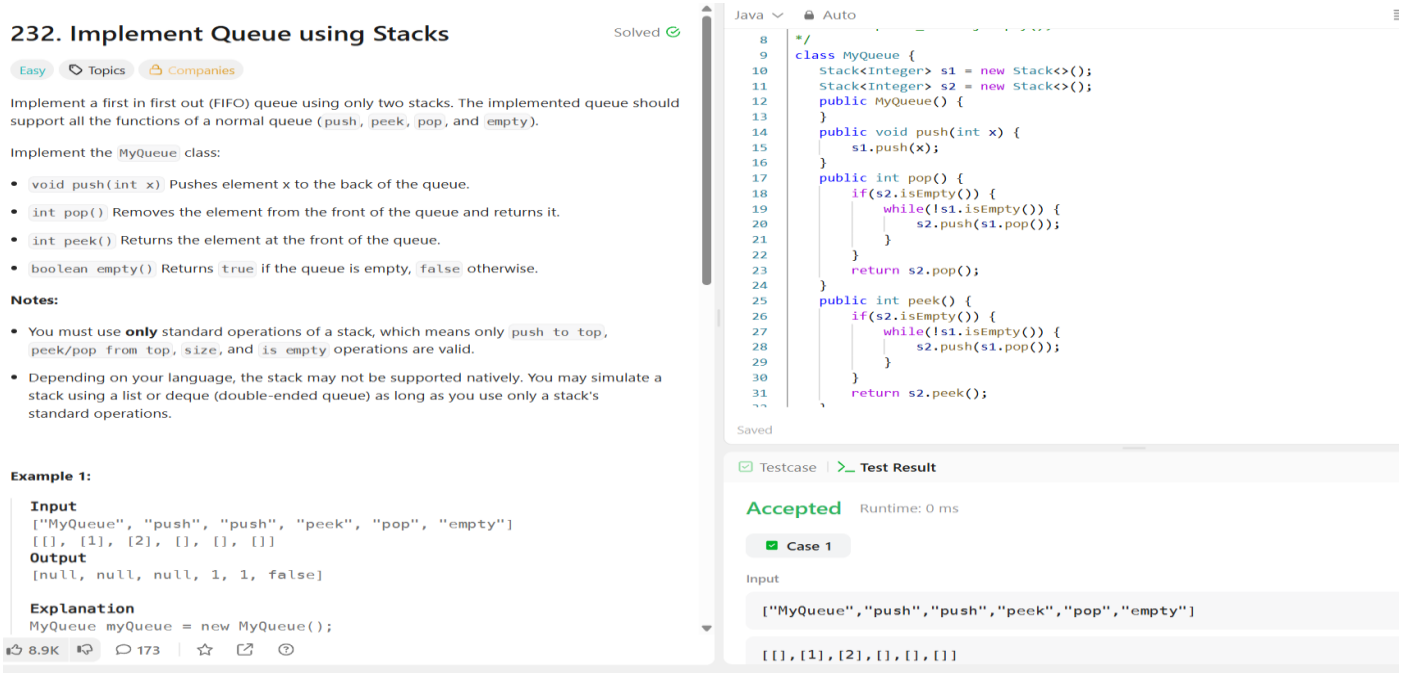
M. Siri Chandana

240A41139

1.Implement Queue using Stack – LC 232

https://leetcode.com/problems/implement-queue-using-stacks/submissions/2189574144/



232. Implement Queue using Stacks

Solved

Easy | Topics | Companies

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (`push`, `peek`, `pop`, and `empty`).

Implement the `MyQueue` class:

- `void push(int x)` Pushes element x to the back of the queue.
- `int pop()` Removes the element from the front of the queue and returns it.
- `int peek()` Returns the element at the front of the queue.
- `boolean empty()` Returns `true` if the queue is empty, `false` otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only `push to top`, `peek/pop from top`, `size`, and `is empty` operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

```
Input
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 1, 1, false]

Explanation
MyQueue myQueue = new MyQueue();
```

8.9K | 173

Java | Auto

```
*/
class MyQueue {
    Stack<Integer> s1 = new Stack<>();
    Stack<Integer> s2 = new Stack<>();
    public MyQueue() {
    }
    public void push(int x) {
        s1.push(x);
    }
    public int pop() {
        if(s2.isEmpty()) {
            while(!s1.isEmpty()) {
                s2.push(s1.pop());
            }
        }
        return s2.pop();
    }
    public int peek() {
        if(s2.isEmpty()) {
            while(!s1.isEmpty()) {
                s2.push(s1.pop());
            }
        }
        return s2.peek();
    }
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

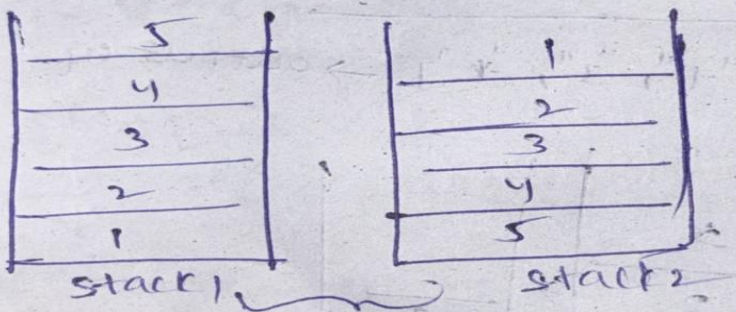
Input

```
["MyQueue","push","push","peek","pop","empty"]
[[],[1],[2],[],[],[]]
```

232. Implement Queue using stack

["MyQueue", "push", "push", "peek", "pop", "empty"]

myQueue → creates new Queue
push → adds element
peek → gives last inserted val
pop → removes last inserted value
empty → returns true if empty.



2 stacks
stack2.push(stack1.peek());
stack1.pop();

y

2. Number of recent calls – LC 933

https://leetcode.com/problems/number-of-recent-calls/submissions/2189580227/

Description | Accepted | Editorial | Solutions | Submissions

933. Number of Recent Calls

Solved

Easy | Topics | Companies

You have a `RecentCounter` class which counts the number of recent requests within a certain time frame.

Implement the `RecentCounter` class:

- `RecentCounter()` Initializes the counter with zero recent requests.
- `int ping(int t)` Adds a new request at time `t`, where `t` represents some time in milliseconds, and returns the number of requests that have happened in the inclusive range `[t - 3000, t]`, that is, the new request plus every earlier request that is no more than `3000` milliseconds older.

It is **guaranteed** that every call to `ping` uses a strictly larger value of `t` than the previous call.

Example 1:

```
Input
["RecentCounter", "ping", "ping", "ping", "ping"]
[[], [1], [100], [3001], [3002]]
Output
[null, 1, 2, 3, 3]

Explanation
RecentCounter recentCounter = new RecentCounter();
recentCounter.ping(1);     // requests = [1], range is [-2999,1], return 1
recentCounter.ping(100);   // requests = [1, 100], range is [-2900,100], return 2
recentCounter.ping(3001);  // requests = [1, 100, 3001], range is [1,3001], return 3
recentCounter.ping(3002);  // requests = [1, 100, 3001, 3002], range
```

947 | 226

Code

Java | Auto

```java
/**
 * Your RecentCounter object will be instantiated and called as such:
 * RecentCounter obj = new RecentCounter();
 * int param_1 = obj.ping(t);
 */
class RecentCounter {
    Queue<Integer> q = new LinkedList<>();
    public RecentCounter() {
    }
    public int ping(int t) {
        q.add(t);
        while(q.peek() < t - 3000) {
            q.poll();
        }
        return q.size();
    }
}
```

Saved

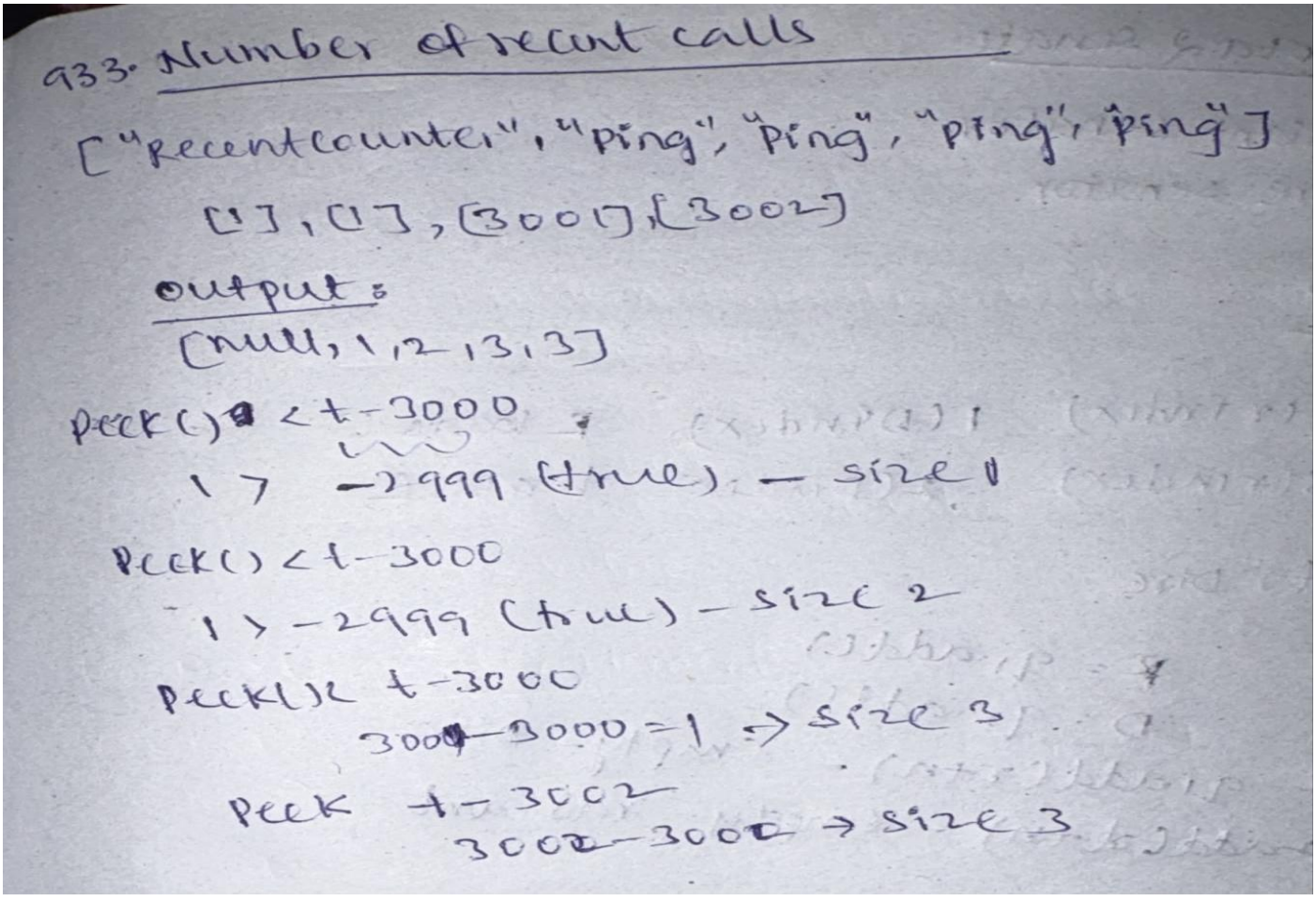
Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

```
["RecentCounter","ping","ping","ping","ping"]
[[],[1],[100],[3001],[3002]]
```



3.Design circular Queue – LC 622

https://leetcode.com/problems/design-circular-queue/submissions/2189591748/

622. Design Circular Queue

Solved

Medium | Topics | Companies

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the `MyCircularQueue` class:

- `MyCircularQueue(k)` Initializes the object with the size of the queue to be `k`.
- `int Front()` Gets the front item from the queue. If the queue is empty, return `-1`.
- `int Rear()` Gets the last item from the queue. If the queue is empty, return `-1`.
- `boolean enQueue(int value)` Inserts an element into the circular queue. Return `true` if the operation is successful.
- `boolean deQueue()` Deletes an element from the circular queue. Return `true` if the operation is successful.
- `boolean isEmpty()` Checks whether the circular queue is empty or not.
- `boolean isFull()` Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

Input

Java | Auto

```
43      }
44      public int Front() {
45          if(count == 0) {
46              return -1;
47          }
48          return arr[front];
49      }
50      public int Rear() {
51          if(count == 0) {
52              return -1;
53          }
54          return arr[rear];
55      }
56      public boolean isEmpty() {
57          return count == 0;
58      }
59      public boolean isFull() {
60          return count == size;
61      }
62  }
```

Saved Ln 53,

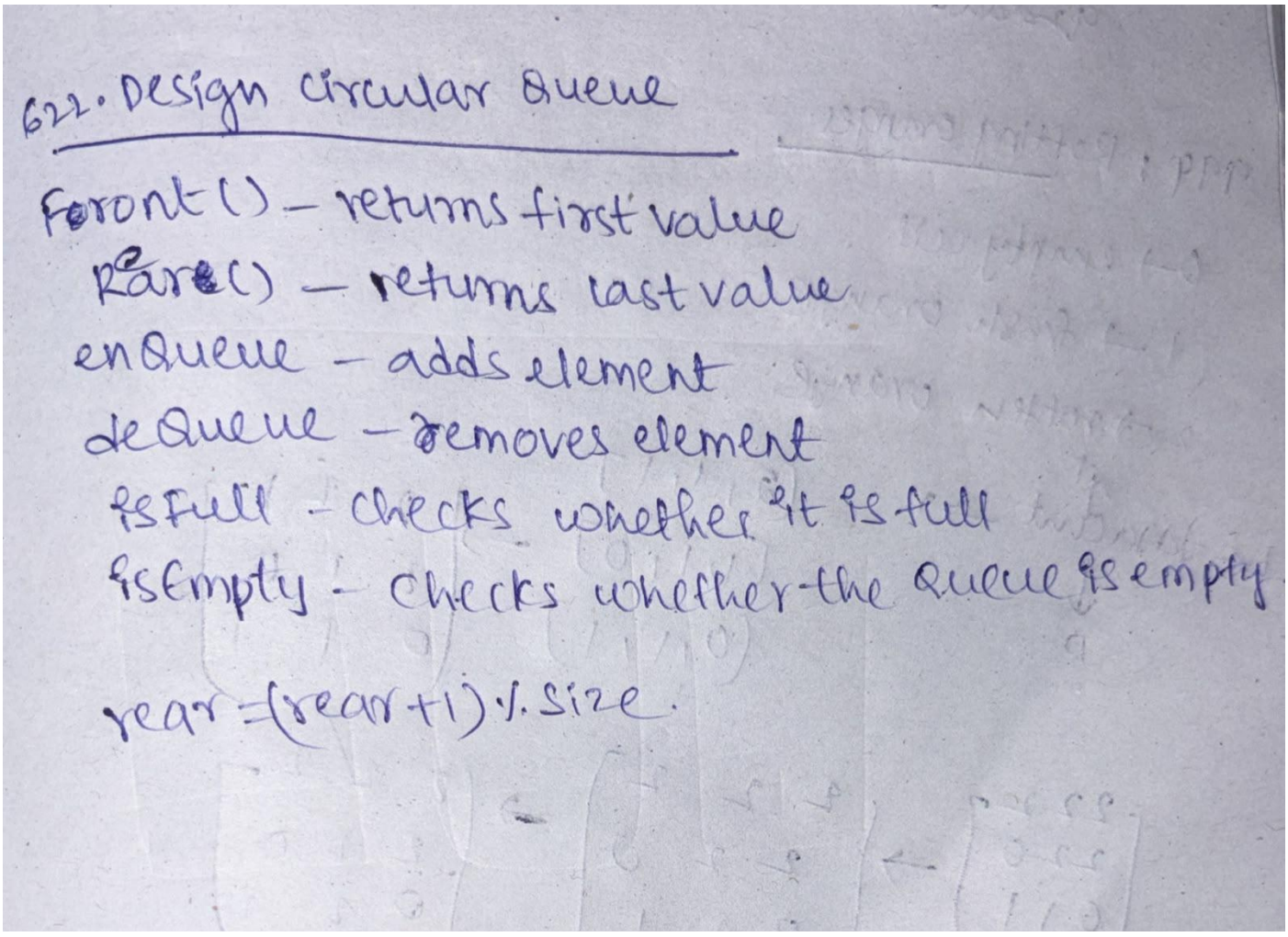
Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

```
["MyCircularQueue","enQueue","enQueue","enQueue","enQueue","Rear","isFull","deQueue","enQueue","Rear"]
```



4. Dota2 senate – LC 649

https://leetcode.com/problems/dota2-senate/submissions/2189598385/

Description | Accepted | Editorial | Solutions | Submissions

649. Dota2 Senate

Solved

Medium | Topics | Companies

In the world of Dota2, there are two parties: the Radiant and the Dire.

The Dota2 senate consists of senators coming from two parties. Now the Senate wants to decide on a change in the Dota2 game. The voting for this change is a round-based procedure. In each round, each senator can exercise **one** of the two rights:

- **Ban one senator's right:** A senator can make another senator lose all his rights in this and all the following rounds.
- **Announce the victory:** If this senator found the senators who still have rights to vote are all from the same party, he can announce the victory and decide on the change in the game.

Given a string `senate` representing each senator's party belonging. The character `'R'` and `'D'` represent the Radiant party and the Dire party. Then if there are `n` senators, the size of the given string will be `n`.

The round-based procedure starts from the first senator to the last senator in the given order. This procedure will last until the end of voting. All the senators who have lost their rights will be skipped during the procedure.

Suppose every senator is smart enough and will play the best strategy for his own party. Predict which party will finally announce the victory and change the Dota2 game. The output should be `"Radiant"` or `"Dire"`.

Example 1:

```
Input: senate = "RD"
Output: "Radiant"
Explanation:
The first senator comes from Radiant and he can just ban the next
```

Code

Java | Auto

```java
for(int i = 0; i < n; i++) {
    if(senate.charAt(i) == 'R') {
        q1.add(i);
    }
    else {
        q2.add(i);
    }
}
while(!q1.isEmpty() && !q2.isEmpty()) {
    int r = q1.poll();
    int d = q2.poll();
    if(r < d) {
        q1.add(r + n);
    }
    else {
        q2.add(d + n);
    }
}
if(q1.isEmpty()) {
    return "Dire";
}
else {
    return "Radiant";
}
}
```

Saved

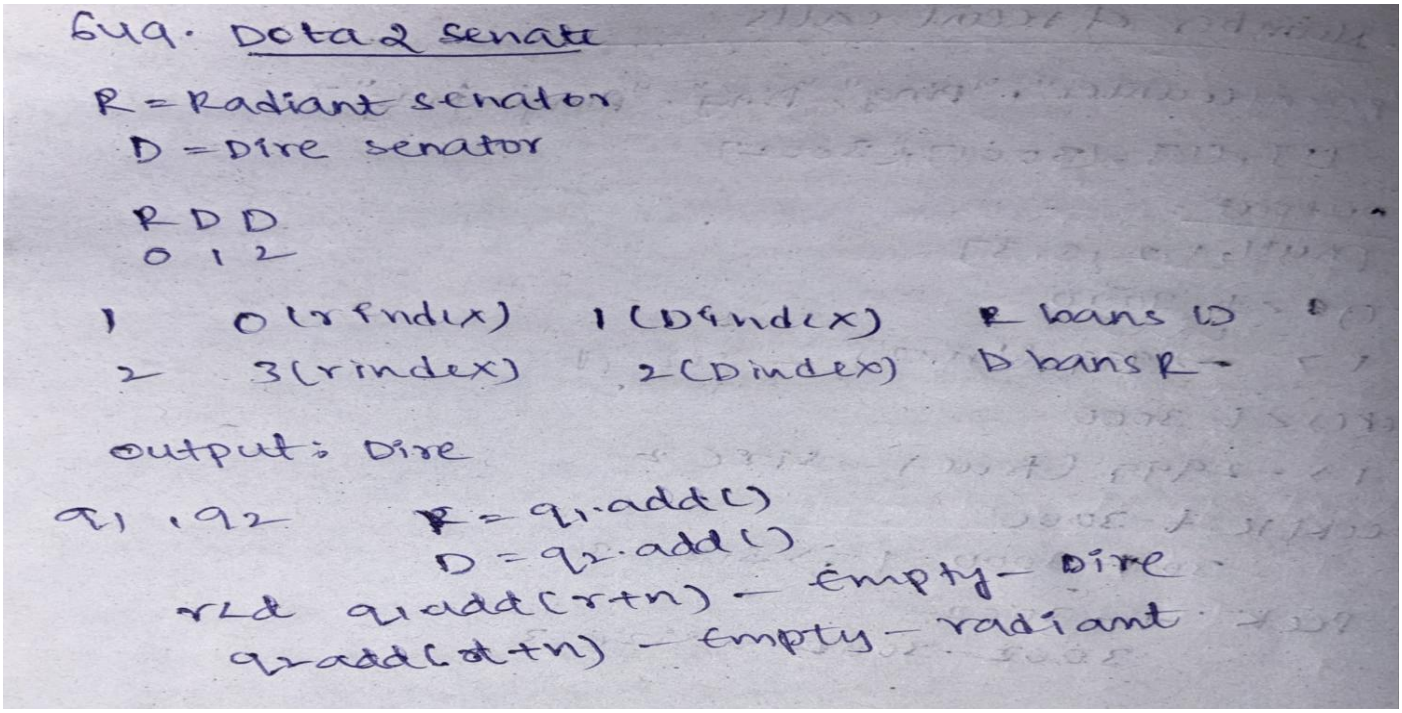
Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

senate =
"RD"



5. Rotting oranges – LC 994

https://leetcode.com/problems/rotting-oranges/submissions/2189604018/

994. Rotting Oranges

Solved

Medium | Topics | Companies

You are given an `m x n` `grid` where each cell can have one of three values:

- `0` representing an empty cell,
- `1` representing a fresh orange, or
- `2` representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return *the minimum number of minutes that must elapse until no cell has a fresh orange.* If *this is impossible, return* `-1`.

Example 1:




```
Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4
```

Example 2:

```
Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0)
```

15.7K 328

Java | Auto

```
28                  int nr = r + row[j];
29                  int nc = c + col[j];
30                  if(nr >= 0 && nc >= 0 &&
31                     nr < n && nc < m &&
32                     grid[nr][nc] == 1) {
33                      grid[nr][nc] = 2;
34                      fresh--;
35                      q.add(new int[]{nr, nc});
36                  }
37              }
38          }
39
40          time++;
41      }
42      if(fresh > 0) {
43          return -1;
44      }
45      return time;
46  }
47 }
```

Saved Ln 47, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

grid =

```
[[2,1,1],[1,1,0],[0,1,1]]
```

